

Security Assessment



Item 23 Internal Report

October 2026

Prepared for ether.fi





Disclaimer

This document is intended for **internal** use only.

Initial Commit Hash	Final Commit Hash
89e9b663	16213cc2

Protocol Scope

- src/across/AcrossSwapModule.sol
- src/enso/EnsoSwapModule.sol
- src/modules/cash/CashEventEmitter.sol
- src/modules/cash/CashModuleCore.sol
- src/modules/cash/CashModuleSetters.sol
- src/modules/cash/CashModuleStorageContract.sol
- src/modules/ModuleLendGatewaySandwich.sol

Low Severity Issues

L-01: Cross-module reentrancy can undercollateralize pending withdrawals

Severity: Low	Impact: Low	Likelihood: Low
Files: src/across/AcrossSwapModule.sol src/enso/EnsoSwapModule.sol src/modules/openocean-swap/OpenOceanSwapModule.sol	Status: Fixed	

Description:

Modules calculate a Safe's available loose balance as:

```
None
available = tokenBalance - pendingWithdrawalAmount;
```

This check is performed before the module approves and calls an external integration, but the checked amount is not reserved for the duration of that external call. Therefore, if the integration can call back after approval is granted but before it pulls the approved tokens, another module action with a separate valid Safe authorization can observe the same tokens as still available.

In the Across origin-swap path, AcrossSwapModule releases the existing withdrawal hold, checks the source-token balance, approves the configured Across periphery, forwards the signed swapData, and resets the approval only after the external call returns. In the affected configuration, the Across periphery exposes multicall, so a signed payload can execute a callback subcall before a later subcall, such as swapAndBridge, pulls the source tokens from the Safe.

During that callback, the attacker can invoke another EtherFi module using a separate valid Safe authorization for the current Safe nonce. The practical path is not reentering the same swap

execution: the swap modules are nonReentrant, and a nested swap that resets the same allowance may also cause the suspended outer call to revert. The viable path is to invoke another module action that creates a new pending withdrawal hold for the same source token.

The callback-created hold is accepted because the source tokens are still in the Safe at that point. The outer Across call then resumes and consumes those same tokens. This leaves the Safe in an under-reserved state:

```
None
token.balanceOf(safe) < pendingWithdrawalAmount
```

The pending withdrawal is now backed by tokens that have already been spent by the outer transaction, so it cannot be processed unless the Safe is later replenished or the request is cancelled.

A related OpenOcean path exists, but only in a narrower form. The normal OpenOcean swap path with SHOULD_CLAIM pulls the source token before downstream execution. However, OpenOceanSwapModule validates the decoded swap description but does not strictly require the router calldata selector to be OpenOcean's swap entry point. If the Safe signs crafted calldata targeting OpenOcean's inherited uniswapV3SwapTo path, a malicious/callback-capable Uniswap V3 output token can reenter during the pool's output-token transfer, before OpenOcean's V3 swap callback pulls the input token from the Safe. That callback can create a pending hold for the input token, after which the V3 callback consumes the same input.

Alternatively, during an Enso swap, an input token's approval callback can invoke another module to create an Across withdrawal hold against loose output-token funds in the safe. Enso's later auto-supply uses its output balance delta without rechecking reservations, so it can supply those held funds into Aave, reaching the aforementioned state of pending hold exceeding the balance of the safe.

Recommendations:

After each external call, enforce a post-condition that the reserved balance is less than the loose balance of the safe.
For the Enso edge case, consider supplying only the loose amount, instead of the balance delta, as the reserved amount may have been changed during the external call.

Customer Response:

Fixed in PR-341

Fix Review:

Fixed

Informational Issues

I-01: Recipient validation is not bound to the destination chain

Description:

AcrossSwapModule and EnsoSwapModule restrict the recipient to the initiating Safe or its paired Cash Safe/Trading Safe. However, the recipient check is performed independently of order.dstChainId:

For an Optimism Cash Safe S, the validation permits either:

- S, which is the expected same-address recipient on supported TopUp chains such as Arbitrum; or

- T, the CREATE3-derived Trading Safe address associated with S, which is intended only for Ethereum Mainnet.

The expected recipient therefore depends on the destination:

Ethereum Mainnet - Derived Trading Safe `T`

Supported TopUp chain - Same-address recipient `S`

Consequently, an order targeting Arbitrum may specify the Ethereum Trading Safe address T, even if no corresponding account exists at that address on Arbitrum.

Recommendations:

Trading safe should be the only supported destination on Ethereum Mainnet.

Customer Response:

Acknowledged

I-02: OpenOceanModule diverges from the PR's policy on spendable assets

Description:

In order to prevent safes from double spending, the new _requiresSolvencyHold validation will require a solvency hold whenever the asset is spendable.

However on the OpenOceanSwapModule, all swaps can be executed atomically, without any delays.

Recommendations:

Consider adding consistent validation on OpenOceanModule as well.

Customer Response:

Acknowledged - *"All open ocean swaps are fully atomic on OP so we don't need a pending hold for these swaps"*

I-03: getDelays is still used by other modules, even though the getWithdrawalDelayForModule introduces a module specific delay

Description:

In AcrossSwapModule and EnsoSwapModule, the getDelays() function has been replaced by getWithdrawalDelayForModule(). However, other modules still use getDelays() in order to determine the withdrawal delay.

The PR mentions that the module-specific delays will be used for trading and explicitly states that:
"Out of scope: StockWithdrawModule keeps its current hold (iTokens are being removed)."

However, if the module specific delay is later applied to a non-trading module, which still uses getDelays, this could potentially cause issues in the future.

Recommendations:

Consider calling _withdrawalDelayForModule within getDelays to accurately determine the withdrawal delay of the module.

Customer Response:

Fixed in 07a46271

Fix Review:

Fix confirmed





Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline. It is helpful for smart contract developers and security researchers during auditing and bug bounties.

Certora also provides services such as auditing, formal verification projects, and incident response.